CookGPT

AI-Powered Personalized Recipe Recommendation System

Rukai Gao Erxing Yang

Washington University in St. Louis

CSE 314A | SP 2025

04/28/2025

Background

For both experienced and inexperienced home cooks, recipe-sharing websites like Food.com are valuable resources. These platforms offer a wide range of recipes to choose from and connect users who share a passion for cooking. However, existing recipe websites often face several challenges:

1. Choice Overload: Most platforms present users with an overwhelming number of options. For those seeking quick and simple recommendations, this abundance can make it difficult to find relevant recipes efficiently.
2. Limited Search Capabilities: The search engines on many recipe websites are often limited to filtering by recipe name or basic ingredients. This is insufficient for inexperienced home cooks who may not know what to make, or for users who want to filter recipes by other criteria such as cooking time or nutritional content.
3. Complex User Interfaces: Even well-designed sites like Food.com, which organize recipes by popularity, ingredients, dietary needs, holidays, cuisine, or season, can overwhelm users with their complexity and numerous filtering options.

In summary, home cooks need a recipe recommendation platform that provides flexible, personalized recommendations without overwhelming them with a complicated user interface or too many choices.

Introduction

To address the gaps found in traditional recipe websites, we developed CookGPT—a command-line, AI-powered recipe recommendation platform. CookGPT is built on a comprehensive dataset of over two million recipes sourced from Food.com, one of the world's largest and most acclaimed recipe-sharing websites.

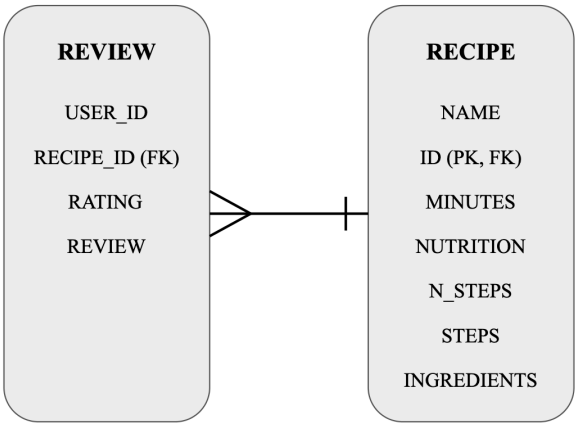
Unlike conventional search engines or complex category systems, CookGPT streamlines the user experience by asking just three simple questions:

1. What food ingredients do you have?
2. Do you have any food allergies?
3. What meal are you looking to prepare?

There are no predefined answer choices; users can respond to these questions in free-form natural language, as if conversing with another person. CookGPT leverages ChatGPT's advanced natural language processing capabilities to understand and extract the user's requirements and preferences from their answers.

Instead of overwhelming users with an exhaustive list of results, CookGPT presents only the top 20 recipe recommendations, each with a concise overview for easy browsing and selection. By providing maximum flexibility and a conversational interface, CookGPT is designed for everyone: from inexperienced home cooks unsure of what to make, to users with dietary restrictions or preferences, to anyone seeking quick, truly personalized meal ideas.

Dataset



Our platform is built upon the Kaggle dataset *"Food.com Recipes and Interactions,"* which contains over 180,000 recipes and 700,000 user reviews, spanning 18 years of uploads and interactions on Food.com (formerly GeniusKitchen). This dataset was originally used in the work by Majumder et al. (2019).

Key datasets and features include:

RAW_recipes.csv (Recipe)

- name: recipe name
- id: unique recipe identifier (PK)
- minutes: estimated cooking time in minutes
- nutrition: list of nutrition information
- n_steps: number of steps
- steps: text description of step-by-step cooking instructions
- ingredients: a list of required ingredients

RAW_interactions.csv (Review)

- user_id: unique user identifier
- recipe_id: foreign key to recipe (FK)
- rating: user rating on a scale of 1-5
- review: text review on a recipe

There is a one-to-many relationship between RAW_recipes.csv and RAW_interactions.csv: each recipe can be associated with multiple reviews from different users

Data Pipeline

Feature Engineering

During the early testing phase of our platform, we identified two significant issues:

1. When users requested a specific meal type, such as dinner, the algorithm sometimes recommended items like milkshakes or ice cream. While these options matched other user preferences (nutrition, cooking time, ingredient availability), they were clearly unsuitable for the requested meal.

2. Food allergies were not initially considered in our recommendations, posing potential health risks and reducing the system's usefulness for users with dietary restrictions.

To address these issues, we implemented a feature engineering step that generates two additional columns, "meal_type" and "allergens", from the existing dataset.

Both features were generated using OpenAI's GPT-4o model via the OpenAI API. Human labeling was not used due to the sheer size of the dataset, over 200,000 recipes, making manual annotation impractical and time-consuming. Algorithmic methods were also set aside for two main reasons:

First, classifying recipes by meal type requires human-like reasoning to interpret context and usage, which traditional algorithms would struggle to achieve without extensive training and semantic analysis. Training a medium-sized language model like BERT for this specific task was both unnecessary and inefficient, given the capabilities of GPT-4o.

Second, for allergen detection, a simple exact-match approach is insufficient. Instead, we aimed to capture indirect relationships, for example, recognizing that ingredients like cake inherently contain major allergens such as eggs and wheat, even if not explicitly listed. GPT-4o's advanced language understanding allows for more nuanced, accurate annotation of potential allergens.

For meal type classification, we prompted GPT-4o to assign each recipe to exactly one of the following categories: breakfast, dinner, dessert, drink, or other. The complete GPT prompt used for this task is provided in Appendix I. The resulting classifications were saved to a separate CSV file containing the recipe ID and its assigned meal type, facilitating easier evaluation of classification accuracy.

For allergen detection, we focused on the eight "major food allergens" identified by the Food Allergen Labeling and Consumer Protection Act of 2004 (FALCPA): milk, eggs, fish, crustacean shellfish, tree nuts, peanuts, wheat, and soybeans. These eight allergens account for approximately 90% of food allergies and serious allergic reactions in the United States. Under FALCPA, foods or ingredients containing a major allergen must be explicitly labeled with the name of the allergen source. The complete GPT prompt for allergen annotation is provided in

Appendix II. The results were similarly stored in a separate CSV file, consisting of recipe IDs and their corresponding list of identified allergens.

Data Cleaning

In the original dataset, the ingredients, nutrition, and steps columns are stored as strings representing Python lists. We used *ast.literal_eval* to safely convert these string representations back into actual Python list objects for further processing.

For the ingredients column, each ingredient name was then converted to lowercase after parsing, ensuring consistency and avoiding issues with case sensitivity during subsequent analysis.

Denormalization

To enable more efficient analysis, we performed denormalization by merging four key tables—recipes, reviews, meal type, and allergens—into a single comprehensive dataset.

- Meal type and Allergens:
  We left-joined the meal_type table onto the main recipes table using recipe ID as the key. The allergens table was similarly left-joined by recipe ID.
- Reviews and Ratings:
  Reviews were grouped by recipe ID and mapped to the corresponding recipe. Ratings were grouped in the same way by recipe ID. We then calculated the average rating for each recipe, normalized the result to a 0–1 scale, and assigned this average rating to the corresponding recipe entry.

After denormalization, the resulting combined recipe table contains the following key features:

Denormalized_Recipe:

- name: recipe name
- id: unique recipe identifier (PK)

- minutes: estimated cooking time in minutes
- nutrition: list of nutrition information
- n_steps: number of steps
- steps: text description of step-by-step cooking instructions
- ingredients: a list of required ingredients
- meal_type: dinner, breakfast, dessert, or other
- allergens: a list of allergen identified
- all_reviews: text of all reviews
- average_ratings: the average of all ratings normalized to a 0-1 scale

Collect User Input

The platform interacts with users through a series of command-line prompts.

1. What ingredients do you have?

   Users are asked to provide a comma-separated list of ingredients they have on hand. Their responses are converted to lowercase, stripped of whitespace, and stored as a Python list of strings.

2. Any food allergies I should keep in mind?

   Users are then prompted to specify any food allergies, again as a comma-separated list, with similar normalization applied to convert the input into a Python list of strings.

3. What kind of meal are you looking to prepare?

   Finally, users are invited to describe the type of meal they wish to prepare in free-form natural language, allowing them to specify preferences and requirements for multiple meals in a single input.

Process User Input

Ingredients

The user's available ingredients are stored as a Python list for subsequent analysis. No additional preprocessing is required.

Allergens

GPT-4o is used to classify the user's allergy input into one or more of the eight major food allergens. This task relies on ChatGPT for the same reason as before: rather than seeking an exact string match, we require human-like reasoning to accurately interpret and classify the user's input. The user's textual input is transformed into a Python list containing one or more of the eight major allergens. The complete GPT prompt for this process can be found in Appendix III.

Meal

GPT-4o is used to transform the user's free-form natural language response into a Python dictionary with the following keys:

- meal_type: one of dinner, breakfast, dessert, drink, or other.
- diet: a list of dietary goals, selected from low sugar, low fat, low sodium, low calorie, low carbohydrate, and high protein. This list is derived from the nutrition column of the recipe dataset, which contains values for calories (absolute value), sugar (% Daily Value), sodium (%DV), fat (%DV), saturated fat (%DV), protein (%DV), and carbohydrate (%DV).
- time: set to 'quick' if the user requests a fast meal; otherwise, this field is left as null.
- ingredients: a list of ingredients the user wants to include in their meal.
- keywords: any additional information identified by ChatGPT as important, but not covered by the other four categories.

Traditional algorithms struggle with this task due to the complexity and nuance of natural language. ChatGPT, with its advanced language understanding, enables the extraction of meaningful and context-aware information from user input. The complete prompt used for this

extraction can be found in Appendix IV.

Filter Recipes

The information extracted in the previous stage is used to filter the recipe database as follows:

- Allergens: All recipes with any allergen in common with the user's identified allergens are excluded, ensuring only safe options are recommended.
- Meal Type: Only recipes that match the user's desired meal type are retained.
- Ingredients: Recipes are kept only if their ingredients column contains at least one of the user-specified ingredients. A substring match, rather than an exact match, is used to increase flexibility and account for variations in ingredient naming.

Rank Recipes

All remaining recipes are ranked using a multi-factor scoring system that incorporates ingredient availability, preparation time, recipe complexity, user ratings, and nutritional value:

- Ingredient Score
  Measures the alignment between the user's available ingredients and the recipe's required ingredients. Calculated as the fraction of required ingredients that the user has available.
- Time Score
  Assigns a score of 0 to recipes that require more than 30 minutes to prepare when the user has requested a "quick" recipe; otherwise, the score is 1.
- Difficulty Score
  Assign a score of 0 to recipes that require more than 10 steps to prepare when the user has specified a preference for an "easy" recipe; otherwise, the score is 1.
- Rating Score
  Based on the average user rating of all corresponding reviews for each recipe, normalized by dividing by 5.

- Nutrition Score

  Following U.S. Food & Drug Administration (FDA) guidelines, 5% Daily Value (DV) or less per serving is considered low, while 20% DV or more is considered high. This standard is applied to all nutrients reported as %DV in the dataset (sugar, sodium, fat, saturated fat, protein, carbohydrate). For calories, since %DV is not provided, we set a daily reference value based on CDC data and USDA calculation tools:

  - Average recommended daily calorie intake: 2,500 calories (based on CDC averages for men and women, assuming 30–60 minutes of moderate activity per day).
  - Low calorie threshold: 125 calories (5% of 2,500).

  In summary, the nutrition rules are as follows:

  - For nutrients measured in %DV:
    - Low: 5% DV or less per serving
    - High: 20% DV or more per serving
  - For calories:
    - Low: 125 calories or less per serving

  The nutrition score is calculated as the proportion of the user's dietary preferences that are met by each recipe. If the user does not indicate any dietary preferences, the nutrition score is set to 1.

All five component scores are carefully designed to fall within the range of 0 to 1. The final recipe score is computed as a weighted sum of these components:

$$final\ score = 0.3\ ing_{score} + 0.3\ nut_{score} + 0.15\ time_{score} + 0.15\ dif_{score} + 0.1\ rating_{score}$$

The weights were determined based on the following considerations:

- Ingredient score is assigned the highest weight (30%) because users typically prioritize using available ingredients to avoid unnecessary shopping.
- Nutrition score also receives the highest weight (30%) to reflect the increasing demand for healthy meal options and personalized dietary goals.

- Time score and difficulty score are each given a moderate weight (15%), emphasizing the importance of quick and easy-to-prepare recipes. These weights are not higher because constraints such as requiring less than 30 minutes of preparation time and fewer than 10 steps are already quite strict.
- Rating score is assigned the lowest weight (10%) since approximately 90% of reviews rate recipes as 4 or 5, making this metric less effective for distinguishing between recipes.

The user is presented with the 20 recipes that achieve the highest overall scores.

Semantic Analysis

For the top 20 recommended recipes, only the key features, recipe name, preparation time, and ingredients, are initially presented. However, during early testing, we found that users also benefit from subjective information to guide their decisions. To address this, we implemented semantic analysis using ChatGPT, extracting the three most meaningful recurring words or short phrases from user reviews that describe each recipe's qualities, experience, or style. This analysis focuses on adjectives, adverbs, and descriptive phrases that capture aspects such as ease of preparation, flavor, portion size, and dietary relevance. To improve responsiveness and reduce user wait time, semantic analysis is performed dynamically: for each recipe, once the keywords are extracted, the recipe is displayed to the user rather than waiting for all 20 analyses to complete. This allows users to begin exploring recommendations without unnecessary delays. The complete prompt used for keyword extraction can be found in Appendix V.

In previous semesters, students conducted semantic analysis using a pre-trained RoBERTa model from HuggingFace. We chose to use GPT-4o for this task instead, based on several key considerations:

- Long-Range Attention and Context Tracking: GPT-4o leverages long-range attention mechanisms, enabling it to analyze patterns across batches of reviews. This allows it to identify not just the most frequent, but the most meaningful and contextually relevant descriptors.
- Phrase-Level Extraction: Unlike models such as RoBERTa, which are generally

optimized for token or word level tasks, GPT-4o can extract and generate meaningful multi-word expressions and phrases.

- Ease of Instruction: GPT-4o can be guided with plain-language instructions, eliminating the need for intensive feature engineering or complex pipeline construction.

Generate Shopping List

If the user opts for a shopping list, both the available ingredients and the required ingredients are provided to ChatGPT, along with instructions to generate a shopping list based on these inputs. While this task might appear straightforward—simply finding ingredients that do not match or partially match between the two lists—testing revealed several challenges. Standard algorithmic approaches often fail to recognize ingredients that can be easily substituted (e.g. brown sugar for white sugar) or made from existing ingredients (e.g. egg whites from whole eggs).

To address this, we instruct ChatGPT to exclude an ingredient from the shopping list if it can be reasonably substituted or covered by something the user already has, which provides a more practical and user-friendly shopping list. The complete prompt used for this process can be found in Appendix VI.

Visualizations

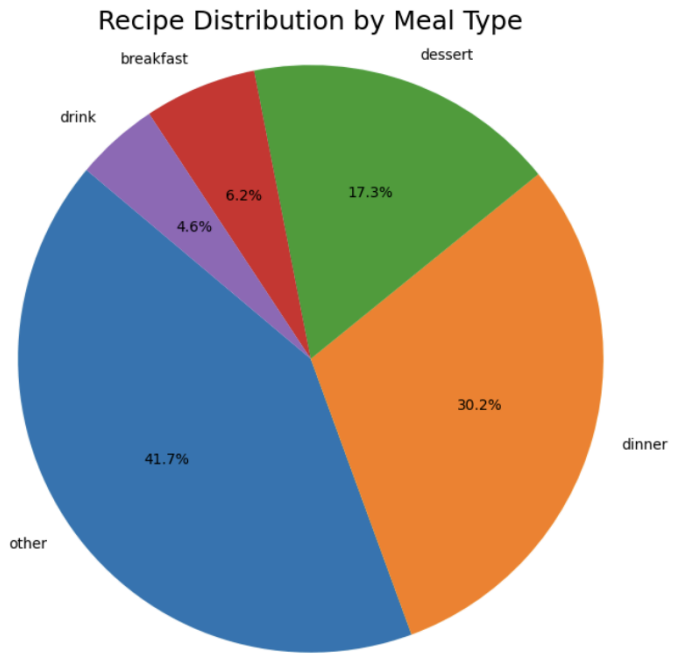


Figure 1. Recipe Distribution by Meal Type

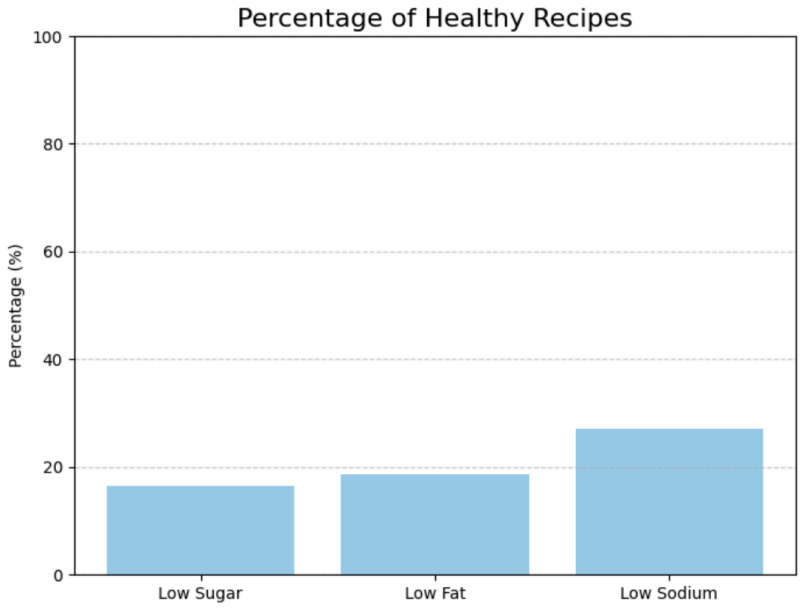


Figure 2. Percentage of Healthy Recipes

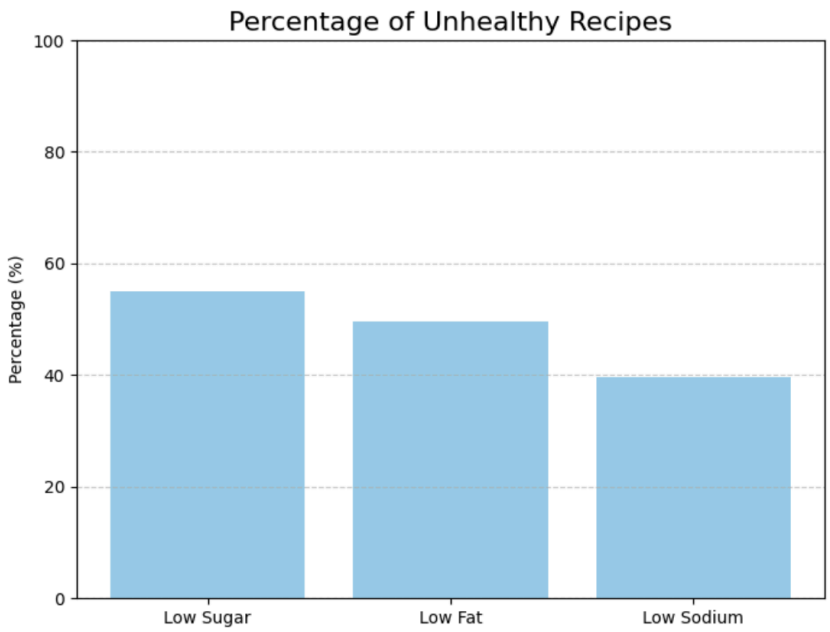


Figure 3. Percentage of Unhealthy Recipes

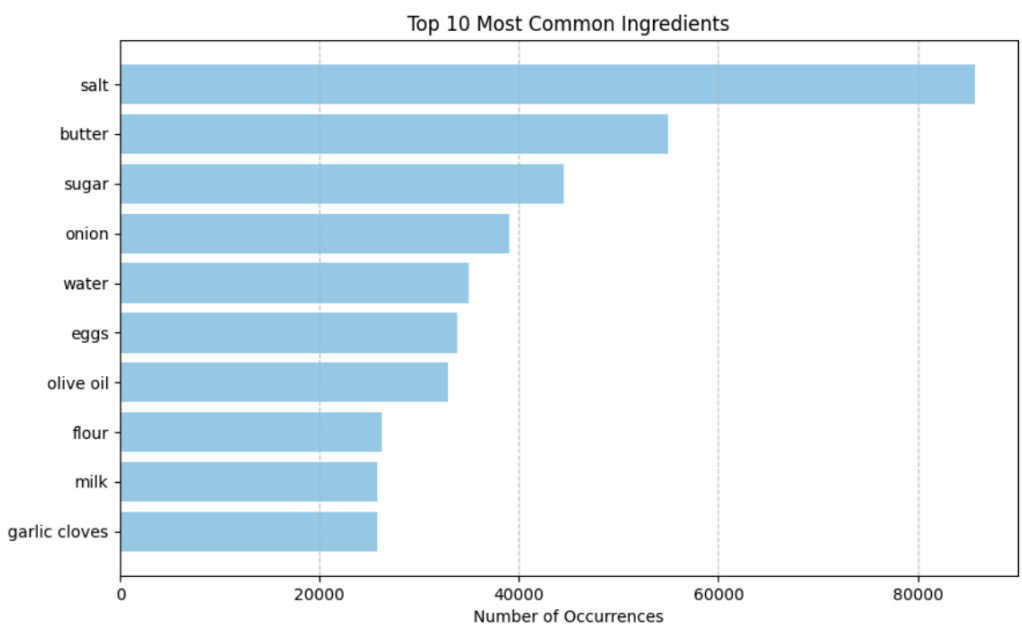


Figure 4. Top 10 Most Common Recipes

Figure 5. Nutrition Radar Chart

Figures 2 and 3 highlight some concerning aspects of dietary patterns in the United States. According to the FDA, a nutrient amount is considered low if it is 5% or less of the Daily Value (DV) per serving, and high if it is 20% or more. Analysis shows that more than half of the recipes are classified as high in sugar and high in fat, while approximately 40% are classified as high in sodium. Maintaining a diet consistently high in sugar and fat significantly increases the risk of weight gain and the development of diabetes. These visualizations emphasize the urgent need to promote healthier dietary choices.

Figure 5 presents an innovative way to visualize the nutritional profile of individual recipes using a radar chart. This format allows users to quickly and intuitively assess the relative risk levels across multiple nutritional factors, offering an accessible overview of a recipe's healthiness at a glance.

Future Improvements

Web Application

While the command-line tool offers flexibility and ease of development, it has certain limitations in terms of user accessibility and experience. To reach a broader audience, developing a web application is a natural next step. However, careful consideration must be given to both the front-end and back-end design, particularly the front end, to balance user flexibility with a concise and intuitive interface. Achieving this balance will be critical to maintaining the platform's simplicity while providing users with the advanced filtering and customization options that set CookGPT apart.

User Profile

With the development of a web application, the introduction of user profiles becomes essential. Users often have consistent dietary preferences and food allergies across different sessions. By storing this information, the platform can streamline the search process and deliver more personalized recommendations.

A potential table structure for user information may include the following fields:

- user_ID: unique identifier for each user (PK)
- user_name: username
- diet: saved dietary preferences (e.g., low sugar, vegetarian)
- allergen: list of known food allergens
- saved_recipe: recipes bookmarked or favorited by the user

Scale & Automation

Our current dataset is based on recipes and reviews shared on Food.com, a community-driven website that receives new content daily. To expand our dataset and keep the platform up-to-date,

one major improvement would be to build an automated data pipeline that collects new recipes and reviews from Food.com and processes them to fit seamlessly into our existing structure.

Scaling the Data Structure

As we scale to a larger dataset, we propose the following updates to the data structure:

- Recipes
  We will shift from RAW_recipes.csv to the denormalized recipe structure. However, to improve efficiency, we propose replacing the "all_reviews" text feature with a new "keywords" feature, storing the three most meaningful recurring words or phrases extracted from reviews. Currently, loading and cleaning the data takes about one minute per run; as the dataset grows, this time could become impractically long without this optimization.
- Reviews
  A new feature "keyword" will be added to the reviews table, storing a Python list of descriptive words or phrases that summarize the recipe's qualities, user experience, or style.

Automating the Data Pipeline

To efficiently manage new data at scale, we propose an automated pipeline that runs daily:

- Web Scraping
  We will scrape newly uploaded recipes and reviews from the "What's Fresh" section of Food.com, which conveniently aggregates all new content. This targeted scraping approach will simplify and streamline the data collection process.
- Data Cleaning
  New recipes will pass through the same cleaning procedures currently used in our data pipeline. Reviews generally require no additional cleaning.
- Feature Engineering (Recipes)
  In the current system, generating "meal_type" and "allergens" using ChatGPT takes approximately four hours each. To avoid long batch processing times, we propose performing feature engineering immediately after new recipes are collected. This

incremental processing approach will ensure that new entries are fully updated without creating large backlogs.

- Feature Engineering (Reviews)
  Newly collected reviews will be fed into ChatGPT to extract the most meaningful descriptive keywords or short phrases. These keywords will then be stored in the new keyword column of the reviews table.
- Update "Keywords"
  We identify all recipes that have received new reviews. For these recipes, we recalculate the three most meaningful recurring words or phrases across all associated reviews and update the "keywords" column with the new list. In other words, we move the Semantic Analysis step from the user-facing pipeline into the daily data update pipeline.

The core idea is to shift as much data preprocessing work as possible into the automated daily pipeline. By doing so, when a user interacts with the platform, we can completely skip time-consuming steps such as Feature Engineering, Data Cleaning, Denormalization, and Semantic Analysis, significantly reducing response time and improving the user experience.

Conclusion

CookGPT is an AI-powered command-line recipe recommendation tool that leverages ChatGPT’s natural language understanding and human-like reasoning to provide personalized recipe suggestions tailored to users' needs. Compared to traditional recipe-sharing websites, CookGPT offers maximum flexibility in how users can phrase their requirements. Users no longer need to navigate complex search engines or sift through overwhelming lists of recipes with redundant labels and categories.

CookGPT is designed with inclusivity and user-friendliness at its core. It carefully considers users with special dietary preferences, restrictions, or food allergies, implementing a robust allergen detection system to ensure safe and suitable recommendations. Our goal is to provide a tool that empowers all users to make confident, informed choices without feeling either overwhelmed by excessive options or restricted by overly narrow selections.

While CookGPT would not be possible without the powerful capabilities of ChatGPT, we apply large language models thoughtfully and selectively. We use ChatGPT only when tasks are too large for manual handling or when human-like reasoning is required—rather than relying on simple string-matching algorithms. In doing so, we achieve a careful balance between search speed and search quality, creating an efficient, safe, and user-centered recipe recommendation experience.

Appendix

Appendix I

```
You are a meal-type classifier.

Classify each recipe title below into **exactly one** of the
following categories:
- breakfast (e.g., eggs, muffins, pancakes, toast)
- dinner (a full/main course meal — includes protein/starch/veg)
- dessert (sweet dishes typically eaten after a meal)
- drink (smoothies, shakes, juices, cocktails, teas, etc.)
- other (side dishes, sauces, dips, dressings, toppings, or things
not eaten alone)

IMPORTANT:
- Only use "dinner" for full meals or clear main courses
- If it's unclear or sounds like a side dish, sauce, or light snack →
use "other"
- Do NOT default to "dinner" unless it's clearly a main meal

Format:
- One label per line
- Use this format: 1. dinner, 2. other, etc.
- Keep the same order and numbering as the titles list

Examples:
- Creamed Spinach → other
- Buttermilk Baked Tomatoes → other
- Roast Chicken with Potatoes → dinner
- Blueberry Pancakes → breakfast
- Strawberry Milkshake → drink
- Chocolate Chip Cookies → dessert
```

```
Titles:
"""
```

Appendix II

```
"You are a food allergen classifier.\n\n"
"Major food allergens (FALCPA): milk, egg, fish (e.g., bass, cod), "
"crustacean shellfish (e.g., crab, lobster, shrimp), tree nuts (e.g.,
almonds, pecans), "
"wheat, peanuts, soybeans.\n"
"- If an ingredient clearly derives from a major allergen (e.g., cake
→ egg, wheat), include it.\n"
"- Only list allergens actually present.\n\n"
"Please respond with **one numbered line per input**, in the
format:\n"
"1. ['egg', 'wheat']\n"
"2. []\n\n"
"Here are the inputs:\n"
```

Appendix III

```
You are an allergy classifier. The user will list food allergies
(possibly in natural language). Your job is to extract all relevant
allergen classes from the following list:

[milk, egg, fish, crustacean shellfish, tree nuts, wheat, peanuts,
soybeans]

Return a Python list of all applicable allergen classes (using only
these names, and only if there is a clear match). Ignore anything
that isn't clearly one of these eight. Do not include any other
explanation or text. Example: if the user input mentions "almonds",
include "tree nuts". If they say "seafood", include both "fish" and
"crustacean shellfish".

User input: "{user_input}"

Your answer must be a valid Python list of strings, e.g. ["milk",
"egg", "peanuts"]
```

Appendix IV

You are a meal preference extractor. Interpret natural language and convert it into a structured Python list of dictionaries, with **one dictionary per requested meal type**.

For each meal type the user requests, create a dictionary with:
- "meal_type": one of ["breakfast", "dinner", "dessert", "drink", "other"]
- "diet": list of dietary goals, choose from: ["low fat", "low sugar", "low sodium", "low calorie", "high protein"]
- "time": "quick" if user wants something fast, else "null"
- "keywords": other descriptive terms (e.g. easy, spicy, creamy)
- "ingredients": a list of specific foods, ingredients, or dish names the user wants to appear in the recipe name (e.g. ["banana", "strawberry", "chocolate"]). If none, use an empty list [].

If a dietary preference or keyword applies only to a specific meal, attach it to the appropriate dictionary. If it applies to all, repeat as needed.

Examples:
User: "I want a healthy dinner with beef and a dessert with banana, low sugar"
Output:
```
[
 {{ "meal_type": "dinner", "diet": ["healthy"], "time": "null", "keywords": [], "ingredients": ["beef"] }},
 {{ "meal_type": "dessert", "diet": ["low sugar"], "time": "null", "keywords": [], "ingredients": ["banana"] }}
]
```

User: "breakfast with eggs, easy and dinner with pork"
Output:
```
[
 {{ "meal_type": "breakfast", "diet": [], "time": "null", "keywords": ["easy"], "ingredients": ["eggs"] }},
 {{ "meal_type": "dinner", "diet": [], "time": "null", "keywords": [], "ingredients": ["pork"] }}
]
```

User: "vegetarian lunch and a low sugar dessert"
Output:
```
[
 {{ "meal_type": "lunch", "diet": ["vegetarian"], "time": "null", "keywords": [], "ingredients": [] }},
```

```
 {{ "meal_type": "dessert", "diet": ["low sugar"], "time": "null",
"keywords": [], "ingredients": [] }}
]

User: "dessert with banana and chocolate, quick and easy"
Output:
[
 {{ "meal_type": "dessert", "diet": [], "time": "quick", "keywords":
["easy"], "ingredients": ["banana", "chocolate"] }}
]

User: "dinner"
Output:
[
 {{ "meal_type": "dinner", "diet": [], "time": "null", "keywords":
[], "ingredients": [] }}
]

User said: "{user_input}"

Your response must be a valid Python list of dictionaries, with one
dictionary per meal type as in the examples above.
```

Appendix V

```
Analyze the following reviews and extract the 3 most meaningful
recurring words or short phrases that describe the recipe's
qualities, experience, or style. Focus especially on adjectives,
adverbs, and descriptive phrases (such as "easy," "quick,"
"delicious," "family style," "kid friendly," "healthy," "comfort
food," etc.). Avoid generic nouns (like "recipe," "dish," "meal") and
exclude filler words.

Reviews:
{joined_reviews}

Only output a comma-separated list of the 3 most relevant descriptive
words or phrases.
```

Appendix VI

```
You are a helpful cooking assistant. The user has these ingredients
in their fridge:
{user_ingredients}
```

The recipe '{recipe_name}' requires these ingredients:
{recipe_ingredients}

For each required ingredient, if it can be reasonably substituted or covered by something the user already has, do NOT include it in the shopping list (for example, 'bananas' is covered by 'banana', and 'brown sugar' is covered by 'sugar'). Only output the ingredients the user definitely still needs to buy.

Return the shopping list as a simple Python list of ingredient names (do not include any explanation or formatting).

Shopping list: